



FRANKFURT UNIVERSITY OF APPLIED SCIENCES
FACULTY OF ENGINEERING – COMPUTER SCIENCE PROGRAM

Traffic Simulation - Java Project

JAVAFX-BASED SUMO TRAFFIC SIMULATION CONTROLLER

Group Number: Tuesday Exercise
(8:15-9:45am) - Group 4

Instructor: Prof. Dr.-Eng. Ghadi
Mahmoudi

Team Members:

1. Minh Luan Vo
2. Lars Michael Keppler
3. Justin Samuel Bons
4. Ali Ajib
5. Abel Solyom

Course: Object-Oriented Programming
in Java

Submission Date:
January 18, 2026

Abstract

This project presents a JavaFX-based graphical user interface for controlling and visualizing traffic simulations in SUMO (Simulation of Urban Mobility). The application allows users to inject vehicles, controlling traffic lights, and interact with the simulation in real time. The system is modular, extensible, and leverages the Libtraci library for SUMO connectivity.

Contents

Declaration of Authorship	2
1 Introduction	3
1.1 Purpose of the Project	3
1.2 Problem Statement	3
1.3 Scope and Approach	3
2 Project Architecture Overview	3
2.1 High-Level Architecture	3
2.2 Concurrency and Responsiveness Strategy	3
2.3 UML Class Diagram	4
3 Program Flow	4
3.1 Entry Point	4
3.2 Stage and Controller Initialization	4
3.3 Simulation Thread and Snapshot Propagation	5
3.4 Illustrative Code Snippet (Startup Wiring)	5
4 File and Component Interaction	6
4.1 Core Files and Responsibilities	6
4.2 How Components Communicate	7
5 Object-Oriented Design Analysis	7
5.1 Encapsulation	7
5.2 Inheritance	7
5.3 Polymorphism	7
5.4 Abstraction	8
6 Data Handling and File Processing	8
6.1 CSV Export	8
6.2 Writing Policy and Frequency	8
6.3 Error Handling Strategy	8
7 Impressive Code and Logic	9
7.1 Vehicle Position Adjustment Before Stop Lines - Luan	9
7.2 Traffic Light Phase Change - Samuel	10
7.3 updateFromSimulation() - Abel	11
7.4 getFilteredVehicleIds() - Abel	11
7.5 createAndInjectVehicle() - Abel	11
7.6 injectVehicle() - Abel	11
7.7 Simulation Thread Management - Ali	13
7.8 CSV Logging of Simulation Steps - Lars M. Keppler	14
8 Conclusion	14

List of Figures

1 UML class diagram of SUMO_Group4 (provided by Abel)	4
---	---

List of Tables

Declaration of Authorship

I hereby declare that the submitted project is my own unaided work or the unaided work of our team. All direct or indirect sources used are acknowledged as references.

I am aware that the project in digital form can be examined for the use of unauthorized aid and in order to determine whether the project as a whole or parts incorporated in it may be deemed as plagiarism. For the comparison of my work with existing sources I agree that it shall be entered in a database where it shall also remain after examination, to enable comparison with future projects submitted. Further rights of reproduction and usage, however, are not granted here.

This work was not previously presented to another examination board and has not been published.

I hereby declare that the submitted project is my own unaided work or the unaided work of our team. All direct or indirect sources used are acknowledged as references.

I am aware that the project in digital form can be examined for the use of unauthorized aid and in order to determine whether the project as a whole or parts incorporated in it may be deemed as plagiarism. For the comparison of my work with existing sources I agree that it shall be entered in a database where it shall also remain after examination, to enable comparison with future projects submitted. Further rights of reproduction and usage, however, are not granted here.

This work was not previously presented to another examination board and has not been published.

Lars Michael Keppler Frankfurt am Main, 23.11.2025

First and last name City, date and signature

Lars Michael Keppler

I hereby declare that the submitted project is my own unaided work or the unaided work of our team. All direct or indirect sources used are acknowledged as references.

I am aware that the project in digital form can be examined for the use of unauthorized aid and in order to determine whether the project as a whole or parts incorporated in it may be deemed as plagiarism. For the comparison of my work with existing sources I agree that it shall be entered in a database where it shall also remain after examination, to enable comparison with future projects submitted. Further rights of reproduction and usage, however, are not granted here.

This work was not previously presented to another examination board and has not been published.

Justin Samuel Bons Frankfurt am Main, 23.11.2025

First and last name City, date and signature

Justin Samuel Bons

I hereby declare that the submitted project is my own unaided work or the unaided work of our team. All direct or indirect sources used are acknowledged as references.

I am aware that the project in digital form can be examined for the use of unauthorized aid and in order to determine whether the project as a whole or parts incorporated in it may be deemed as plagiarism. For the comparison of my work with existing sources I agree that it shall be entered in a database where it shall also remain after examination, to enable comparison with future projects submitted. Further rights of reproduction and usage, however, are not granted here.

This work was not previously presented to another examination board and has not been published.

Ali Ajib Frankfurt am Main, 23.11.2025

First and last name City, date and signature

Ali Ajib

I hereby declare that the submitted project is my own unaided work or the unaided work of our team. All direct or indirect sources used are acknowledged as references.

I am aware that the project in digital form can be examined for the use of unauthorized aid and in order to determine whether the project as a whole or parts incorporated in it may be deemed as plagiarism. For the comparison of my work with existing sources I agree that it shall be entered in a database where it shall also remain after examination, to enable comparison with future projects submitted. Further rights of reproduction and usage, however, are not granted here.

This work was not previously presented to another examination board and has not been published.

Abel Solyom Duisburg, 24.11.25

First and last name City, date and signature

Abel Solyom

I hereby declare that the submitted project is my own unaided work or the unaided work of our team. All direct or indirect sources used are acknowledged as references.

I am aware that the project in digital form can be examined for the use of unauthorized aid and in order to determine whether the project as a whole or parts incorporated in it may be deemed as plagiarism. For the comparison of my work with existing sources I agree that it shall be entered in a database where it shall also remain after examination, to enable comparison with future projects submitted. Further rights of reproduction and usage, however, are not granted here.

This work was not previously presented to another examination board and has not been published.

Minh Luan Vo Frankfurt am Main, 23.11.2025

First and last name City, date and signature

Minh Luan Vo

We have used AI tools (i.e. ChatGPT) for research purposes. That is, we have used them to get ideas about libraries such as JavaFX or understanding errors.

1 Introduction

1.1 Purpose of the Project

SUMO_Group4 is a Java application that provides an interactive graphical front-end for monitoring and manipulating a live traffic simulation executed by SUMO (Simulation of Urban Mobility). The software integrates a JavaFX-based user interface with the SUMO runtime through the LibTraCI API, allowing the user to observe simulation state and trigger control actions during execution.

1.2 Problem Statement

Although SUMO provides its own graphical tools, course-oriented projects often require a custom application that demonstrates object-oriented encapsulation of domain concepts (vehicles, lanes, traffic lights) and the orchestration of an external simulation engine from Java. The primary problem addressed by this project is therefore the creation of a maintainable, modular Java program that can:

- connect to a SUMO scenario,
- advance the simulation step-by-step,
- synchronize simulation state with local OOP models,
- provide a responsive GUI for visualization and runtime control.

1.3 Scope and Approach

The scope is limited to live interaction with a SUMO scenario configured through project resources. The design adopts a controller-driven architecture: a central `SimulationController` orchestrates a `SumoConnection` and delegates entity-level responsibilities to dedicated controllers (vehicles, traffic lights, lanes). UI rendering is decoupled from stepping through the simulation via a background thread and a snapshot-based update mechanism.

2 Project Architecture Overview

2.1 High-Level Architecture

At a high level, the system can be summarized as four cooperating layers:

1. **Presentation Layer (JavaFX):** Main loads the FXML view, while GUI serves as the JavaFX controller handling user input and UI updates.
2. **Simulation Orchestration Layer:** `SimulationController` coordinates simulation stepping, entity management, and high-level operations (spawn, stress test, traffic light changes).
3. **Domain Layer (Controllers and Models):** entity state is represented by model classes (e.g., `VehicleModel`, `TrafficLightModel`, `LaneModel`) which are managed and synchronized by dedicated controllers.
4. **Infrastructure Layer (External Integration):** `SumoConnection` wraps LibTraCI calls to start/step/-close SUMO.

2.2 Concurrency and Responsiveness Strategy

A key architectural aspect is the separation of simulation stepping from UI rendering:

- GUI spawns a background simulation thread (`startSimulationThread`) that repeatedly advances the simulation by calling `SimulationController.singleStep()`.

- Each simulation iteration builds a **SimSnapshot** containing vehicle UI states, traffic light UI states, and aggregate metrics.
- The JavaFX **AnimationTimer** reads the latest snapshot and updates JavaFX nodes and charts on the JavaFX application thread.

This design reduces UI stutter: the simulation can progress even if rendering or chart updates are throttled.

2.3 UML Class Diagram

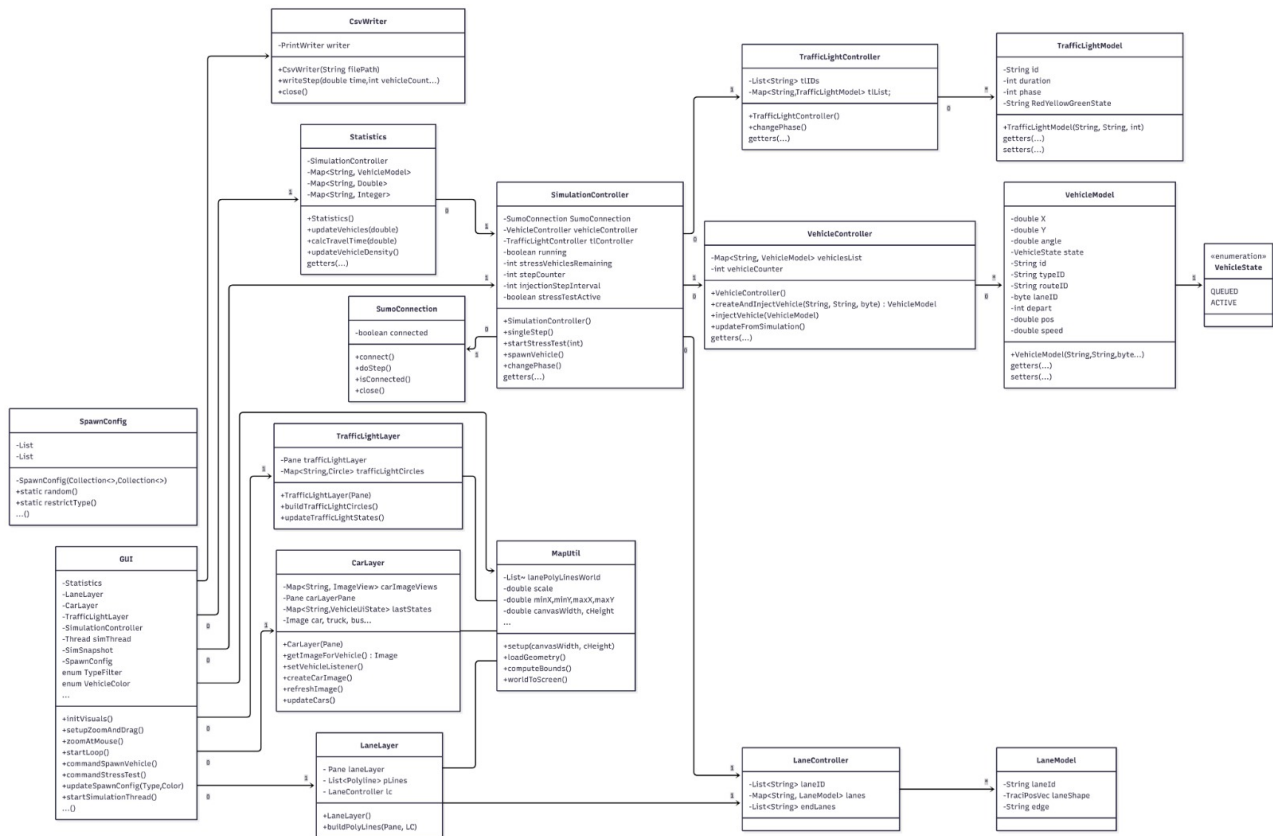


Figure 1: UML class diagram of SUMO_Group4 (provided by Abel)

3 Program Flow

3.1 Entry Point

The program is launched through **Launcher.main**, which delegates to JavaFX by invoking:

- **Application.launch(Main.class, args)**

The actual runtime initialization then starts in:

- **Main.start(Stage)**

3.2 Stage and Controller Initialization

The startup flow in **Main.start** is as follows:

1. Load **GUI-view.fxml** via **FXMLLoader**.

2. Create a `Scene` and show the `JavaFX Stage`.
3. Obtain the `GUI` controller instance from the loader.
4. Create a `SimulationController`.
5. Inject it into `GUI` using:
 - `gui.setSimulationController(simController)`
6. Register a close-request handler asking whether the CSV file `simulation.csv` should be kept or deleted.

3.3 Simulation Thread and Snapshot Propagation

Once `GUI.setSimulationController` is called, UI initialization begins and the runtime is established:

1. `GUI.initVisuals()` instantiates rendering layers and prepares UI controls such as filters, selectors, and charts.
2. `GUI.startLoop()` starts two main processes:
 - a background simulation thread (`startSimulationThread`)
 - a `JavaFX AnimationTimer` that reads the latest snapshot and updates visuals
3. In each simulation loop iteration:
 - Pending actions from the UI are executed using a lock-free `ConcurrentLinkedQueue<Runnable>`, for example:
 - spawn vehicle
 - stress test
 - traffic light phase change
 - The simulation advances one step via `simController.singleStep()`.
 - Models are synchronized using:
 - `VehicleController.updateFromSimulation()`
 - `TrafficLightController.updateTLModel()`
 - Aggregate statistics are updated using:
 - `Statistics.updateVehicles()`
 - `Statistics.updateAndGetAverageTravelTime()`
 - `Statistics.updateTrafficLights()`
 - A `SimSnapshot` is produced for the UI.
 - A CSV row is written approximately once per simulated second.

3.4 Illustrative Code Snippet (Startup Wiring)

```
FXMLLoader fxmllLoader = new FXMLLoader(Main.class.getResource("GUI-view.fxml"));
Scene scene = new Scene(fxmllLoader.load());
stage.setScene(scene); stage.show();

GUI gui = fxmllLoader.getController();
SimulationController simController = new SimulationController();
gui.setSimulationController(simController);
```

4 File and Component Interaction

4.1 Core Files and Responsibilities

The system centers around `GUI` and `SimulationController`. Key files are:

- **Launcher**: JavaFX launcher with `main()` method.
- **Main**: configures JavaFX stage, loads FXML, wires controllers, and manages shutdown (optionally deleting CSV).
- **GUI**: JavaFX controller; handles:
 - UI controls and rendering layers
 - event handlers
 - simulation thread
 - snapshot-driven UI updates
- **SimulationController**: orchestrates simulation; holds:
 - `SumoConnection`
 - `VehicleController`
 - `TrafficLightController`
 - `LaneController`

Provides high-level GUI functions.
- **SumoConnection**: wrapper for LibTraCI to start, step, and close SUMO.
- **VehicleController**: manages vehicles:
 - creation, injection, update, filtering
 - stores `Map<String, VehicleModel>` with thread-safe collections
- **TrafficLightController**: manages traffic lights:
 - discovery and grouping
 - updates `TrafficLightModel`
 - runtime phase changes via SUMO program logic
- **LaneController**: manages lanes:
 - discovery of lanes
 - stores `LaneModel` metadata
 - provides printable lanes and endpoint lists for routing
- **Statistics**: computes metrics:
 - average speed
 - average travel time
 - congestion detection
 - traffic light color counts
- **CsvWriter**: writes periodic simulation observations to `simulation.csv`.

4.2 How Components Communicate

Communication relies on method calls and shared model objects:

- **GUI** → **SimulationController**:
 - triggers domain actions (spawn, stress test, phase change)
 - enqueues tasks into `actionQueue`
- **SimulationController** → **SumoConnection**:
 - starts SUMO
 - advances simulation steps
- **SimulationController** → domain controllers:
 - Vehicles: **VehicleController** creates and synchronizes **VehicleModel**
 - Traffic lights: **TrafficLightController** synchronizes **TrafficLightModel**
 - Lanes: **LaneController** provides metadata and endpoint lists
- **Statistics** depends on **SimulationController** to access:
 - `getVehiclesMap()`
 - `getTlList()`

5 Object-Oriented Design Analysis

5.1 Encapsulation

Encapsulation is applied consistently in the model classes:

- **VehicleModel**, **TrafficLightModel**, and **LaneModel** store state in private fields and expose access via getters and setters.
- The controllers provide higher-level operations (e.g., `VehicleController.createAndInjectVehicle`) so that the GUI does not manipulate LibTraCI calls directly.

A limitation in encapsulation is that some domain constraints are not enforced at the model boundary (e.g., the ability to set arbitrary string identifiers); validations are mostly placed in controller logic.

5.2 Inheritance

There is no custom inheritance hierarchy among the traffic domain classes. Inheritance is present primarily through framework usage:

- **Main** inherits from `javafx.application.Application`, which is a standard pattern in JavaFX applications.

The project intentionally favors composition (controllers holding models) over inheritance for domain concerns.

5.3 Polymorphism

Polymorphism in the classical OOP sense (interfaces and overriding in domain classes) is limited. However, functional polymorphism is used through:

- **Runnable** tasks stored in a queue (`ConcurrentLinkedQueue<Runnable>`) represent heterogeneous UI intentions executed by the simulation thread.
- JavaFX callbacks (event handlers, listeners) are registered dynamically and execute different behaviors based on user actions.

5.4 Abstraction

Abstraction is expressed through role separation and wrappers:

- `SumoConnection` abstracts the LibTraCI simulation control operations (`connect()`, `doStep()`, `close()`), preventing infrastructure details from spreading into domain controllers and GUI logic.
- The project uses controller classes as abstractions over entity operations, reducing coupling between UI and external libraries.

6 Data Handling and File Processing

6.1 CSV Export

The project automatically generates a file `simulation.csv` through `CsvWriter`. The file is overwritten at startup and is written in a semicolon-separated format with a header:

- `time`
- `vehicleCount`
- `avgSpeed`
- `congestionPresent`
- `redLights`, `yellowLights`, `greenLights`

6.2 Writing Policy and Frequency

Within `GUI.startSimulationThread()`, a CSV row is written when simulated time has advanced by at least one second since the last write. This creates a time-series dataset that is suitable for post-processing and performance analysis.

6.3 Error Handling Strategy

Error handling is primarily defensive:

- `SumoConnection.doStep()` checks for a valid connection and throws `IllegalStateException` if absent.
- Most interaction points (UI actions, simulation loop body, file initialization) catch generic `Exception` and log failures using `Log4j`.

This approach favors runtime robustness and continuous operation, although it can obscure specific failure causes if exceptions are overly generalized.

7 Impressive Code and Logic

7.1 Vehicle Position Adjustment Before Stop Lines - Luan

```
Point2D javaPos = MapUtil.worldToScreen(new Point2D(vs.x(), vs.y()));
double w = carView.getFitWidth();
double h = carView.getFitHeight();
double angle = vs.angle();
carView.setRotate(angle);
double radAngle = Math.toRadians(angle);
double offsetX = h / 2 * Math.sin(radAngle);
double offsetY = -(h / 2) * Math.cos(radAngle);
carView.setLayoutX(javaPos.getX() - w / 2.0 - offsetX);
carView.setLayoutY(javaPos.getY() - h / 2.0 - offsetY);
```

Explanation and Logic:

- **Purpose:** Adjust vehicle graphics to visually stop correctly before traffic lights.
- **Step-by-step Logic:**

1. Convert SUMO world coordinates to screen coordinates.
2. Retrieve vehicle image width/height and heading angle.
3. Rotate vehicle image to match simulation heading.
4. Compute offsets using trigonometry to align vehicle front with stop line:

$$offsetX = \frac{h}{2} \cdot \sin(\theta), \quad offsetY = -\frac{h}{2} \cdot \cos(\theta)$$

5. Set layout coordinates adjusting for image size and offsets.

- **Why It Is Smart:**

- Visual accuracy for vehicles at intersections.
- GUI-only adjustment avoids altering SUMO logic.
- Uses mathematically correct offsets for heading and vehicle size.
- Compact and maintainable code.

7.2 Traffic Light Phase Change - Samuel

```
public void changePhase(double duration, String GroupID) {
    for (String id : tlIds) {
        TrafficLightModel tl1 = tlList.get(id);
        if (tl1.getGroupID().equals(GroupID)) {
            int currentPhase = TrafficLight.getPhase(tl1.getId());
            if (currentPhase == 0) {
                tl1.setDuration(duration);
                var tllogic = TrafficLight.getAllProgramLogics(tl1.getId()).
                    get(0);
                tllogic.getPhases().get(2).setDuration(duration);
                TrafficLight.setProgramLogic(tl1.getId(), tllogic);
                tl1.setPhase(2);
                TrafficLight.setPhase(tl1.getId(), 2);
            } else if (currentPhase == 2) {
                tl1.setDuration(duration);
                var tllogic = TrafficLight.getAllProgramLogics(tl1.getId()).
                    get(0);
                tllogic.getPhases().get(0).setDuration(duration);
                TrafficLight.setProgramLogic(tl1.getId(), tllogic);
                tl1.setPhase(0);
                TrafficLight.setPhase(tl1.getId(), 0);
            }
        }
    }
}
```

Explanation and Logic:

- **Purpose:** Change the phase of a traffic light group and persist the next phase duration.
- **Step-by-step Logic:**
 1. Iterate all traffic light IDs.
 2. Select lights in the specified group.
 3. Check current phase (green=0, red=2).
 4. Update next phase duration in local model.
 5. Retrieve program logic from SUMO.
 6. Modify duration of target phase.
 7. Push updated logic back to SUMO.
 8. Update current phase in model and SUMO.
- **Why It Is Smart:**
 - Encapsulation: GUI and simulation code do not manipulate SUMO directly.
 - Persistent Update: Phase duration changes are enforced in SUMO.
 - Group Control: Supports coordinated changes across multiple lights.
 - Robustness: Handles both starting phases consistently.
 - Readability: Clear separation of model update, SUMO update, and phase switch.

7.3 updateFromSimulation() - Abel

```
// Pseudocode summary
updateFromSimulation() {
    // Throttle updates using UI_UPDATE_NANOS
    // Fetch live vehicle IDs
    // Update VehicleModel objects (position, speed, heading, lane)
    // Remove finished vehicles
}
```

Logic and Highlights:

- Keeps local vehicle state synchronized with SUMO.
- Uses throttling to balance performance and UI responsiveness.
- Removes finished vehicles to maintain memory and state integrity.

7.4 getFilteredVehicleIds() - Abel

```
// Pseudocode summary
getFilteredVehicleIds() {
    // Filter VehicleModel objects by type, color, activity
    // Return matching vehicle IDs
}
```

Logic and Highlights:

- Provides filtered vehicle IDs for the GUI.
- Decouples visualization from raw simulation data.

7.5 createAndInjectVehicle() - Abel

```
// Pseudocode summary
createAndInjectVehicle() {
    // Assign unique ID
    // Get SUMO simulation time
    // Prepare VehicleModel
    // Call injectVehicle()
}
```

Logic and Highlights:

- Prepares vehicles in Java before injecting into SUMO.
- Ensures unique IDs and correct timestamping.

7.6 injectVehicle() - Abel

```
// Pseudocode summary
injectVehicle(vehicle) {
    try {
        // Send vehicle data to SUMO
        // Save in vehiclesList
    } catch (Exception e) {
        // Log warning
    }
}
```

Logic and Highlights:

- Bridges Java VehicleModel to SUMO engine.
- Robust error handling avoids crashes due to invalid injection.

7.7 Simulation Thread Management - Ali

```
private void startSimulationThread() {
    running = true;
    simThread = new Thread(() -> {
        CsvWriter csv = null;
        try {
            csv = new CsvWriter("simulation.csv");
        } catch (Exception e) {
            LOG.error("Failed to initialize CSV: " + e.getMessage());
            e.printStackTrace();
        }

        while (running) {
            try {
                Runnable task;
                while ((task = actionQueue.poll()) != null) {
                    try { task.run(); } catch (Exception ex) { LOG.error(ex.getMessage()); }
                }
                simController.singleStep();
                double time = simController.getTime();
                Collection<String> ids = simController.getVehicleController
                    ()
                    .getFilteredVehicleIds(currentTypeFilter,
                        currentColorFilter);
                List<VehicleUiState> states = new ArrayList<>(Math.max(1,
                    ids.size()));
            }
        }
    });
}
```

Logic and Highlights:

- Runs simulation in a separate thread to keep GUI responsive.
- Dispatches UI actions asynchronously via a queue.
- Advances SUMO one step at a time.
- Prepares filtered vehicle states for rendering.
- Includes error handling for robustness.

7.8 CSV Logging of Simulation Steps - Lars M. Keppler

```
/**
 * Writes a single row of simulation data to the CSV file.
 */
public void writeStep(double time, int vehicleCount, double avgSpeed,
    boolean congestionPresent, Map<String, Integer> trafficLightStates) {
    try {
        int r = trafficLightStates.getDefault("R", 0);
        int y = trafficLightStates.getDefault("Y", 0);
        int g = trafficLightStates.getDefault("G", 0);
        writer.printf("%.2f;%d;%.3f;%b;%d;%d;%d\n", time, vehicleCount,
            avgSpeed, congestionPresent, r, y, g);
    } catch (Exception e) {
        System.err.println("Error writing CSV step at time " + time + ": " +
            e.getMessage());
    }
}
```

Explanation and Logic:

- Records one simulation step into CSV, including vehicle counts, speed, congestion, and traffic light states.
- Uses `getDefault` to avoid null pointer issues.
- Try-catch prevents CSV errors from crashing simulation.
- Produces structured, time-series data for analysis and visualization.

8 Conclusion

The SUMO_Group4 project demonstrates a coherent object-oriented architecture for integrating an external simulation engine (SUMO) into a JavaFX desktop application. The key strengths lie in its controller-based separation of concerns, the encapsulation of simulation entities into model classes, and the snapshot-driven concurrency design that maintains UI responsiveness while stepping the simulation in real time.

Potential improvements, consistent with the current implementation, include introducing stronger interface-based abstractions for controllers and data providers, refining exception handling to be more specific, and externalizing configuration elements such as route sets and endpoint-lane lists to resource files. These changes would further enhance maintainability, testability, and adaptability without changing the fundamental architecture.

The End

Thank you for reading.